HBA（Hierarchical Bundle Adjustment）算法分析

> 基于 hku-mars/HBA 源码（RAL 2023）的完整技术解析。涵盖算法原理、分层机制、局部 BA 窗口、体素 octree、复杂度分析。

目录

1. 核心问题：PGO 的局限性
2. 算法总览
3. 滑动窗口 Local BA
4. 体素 octree：平面特征提取
5. 分层金字塔：层间压缩与传播
6. 完整数据流
7. 复杂度分析：为什么比全局 BA 快
8. 关键参数表
9. 相关工作的完整谱系
10. 源码关键函数速查

一、核心问题：PGO 的局限性

回环检测找到"我来过这里"之后，传统做法是**位姿图优化（PGO）**——只优化关键帧之间的相对位姿约束。但 PGO 有个根本局限：

```
PGO 优化后：位姿收敛了，但点云地图仍有"鬼影"（ghosting）
          → 墙面错层、建筑边缘模糊
          → 因为 PGO 不直接优化点云一致性
```

HBA 的目的：在 PGO 之后再做一次分层 BA，直接优化点云的几何一致性，但通过分层策略避免全局 BA 的巨大计算量。

二、算法总览

2.1 核心思想

把大 BA 切成小 BA 分别求解，再用 PGO 把局部修正传播到全局。

```
输入：PGO 后的初始位姿 + 各帧点云

  Layer 0: 1000 帧原始 PCD     → 滑动窗口 BA → 200 个合并关键帧
  Layer 1: 200 个合并关键帧     → 滑动窗口 BA → 40 个合并关键帧
  Layer 2: 40 个合并关键帧      → 全局 BA（无窗口）→ 优化后位姿
                                        |
                                        ▼
                                PGO 传播 → 全部 1000 帧
```

2.2 三种"分层"维度

维度	结构	是不是树
体素剖分 (recut)	八叉树，递归二分到 `layer_limit`	**是，正宗 octree**
层间金字塔	线性序列压缩链（类似图像金字塔）	不是，是单向压缩
窗口划分	平铺并行，无层级关系	不是
PGO 全局传播	因子图	不是，是图

三、滑动窗口 Local BA

3.1 窗口参数

```
#define WIN_SIZE 10   // 每个局部 BA 窗口包含 10 帧
#define GAP 5         // 相邻窗口起始位置间隔 5 帧
```

3.2 窗口排布与并行

```
原始帧:  0   1   2   3   4   5   6   7   8   9  10  11  12  13  14 ...
         ├──────── Win 0 ────────┤
                 ├──────── Win 1 ────────┤
                         ├──────── Win 2 ────────┤
         ← GAP=5 →           ← GAP=5 →

线程0 处理 Win 0   |  线程1 处理 Win 1   |  线程2 处理 Win 2
```

- 窗口之间有 **50% 重叠**（WIN_SIZE=10, GAP=5），保证层间连续性
- 同一帧会出现在两个相邻窗口中被分别优化，最终取哪个窗口的结果由 `update_next_layer_state` 决定

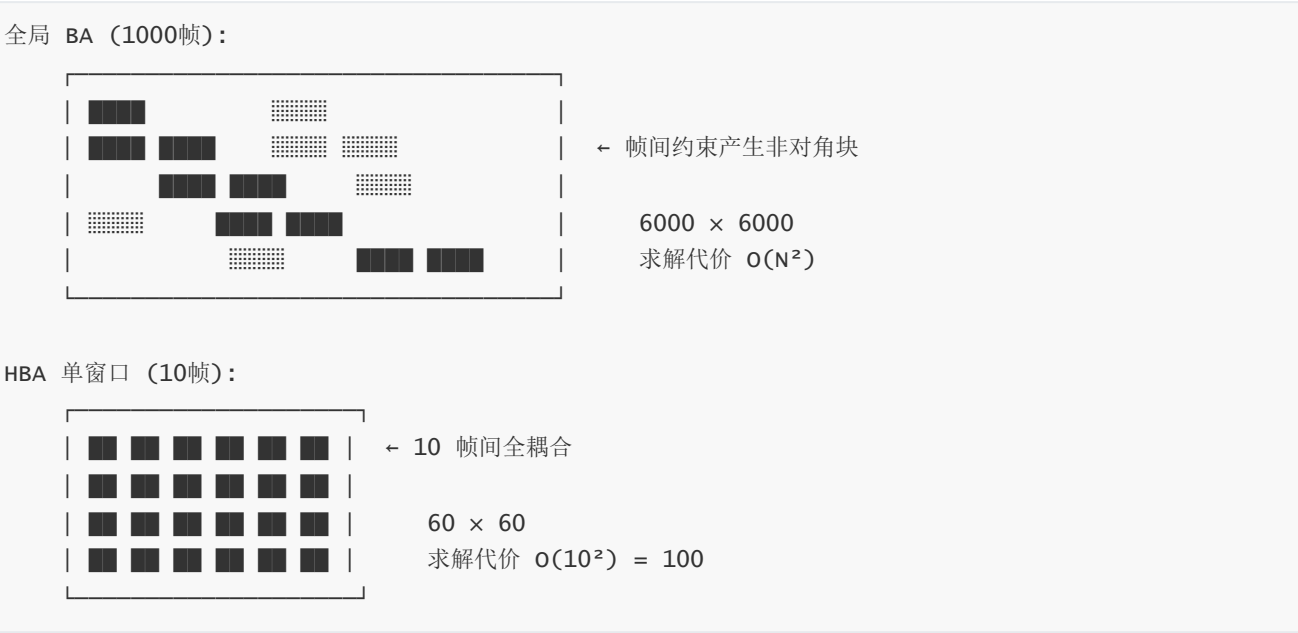
3.3 每个窗口内的优化流程

```
// parallel_comp() — hba.cpp
for (int iter = 0; iter < layer.max_iter; iter++) {
    // 1. 将 Win 内 10 帧点云用当前估计位姿变换到世界系
    // 2. cut_voxel(): 按 voxel_size 分配到体素，构建 octree
    // 3. recut():     递归剖分 octree，提取平面特征
    // 4. tras_opt(): 收集所有平面体素 → 构建 Hessian
    // 5. damping_iter(): LM 优化 10 帧位姿（10×6=60 变量）
    // 6. 检查收敛: |res_prev - res_cur| / |res_cur| < 5%
}

// 优化收敛后：将这 10 帧点云合并为 1 个"关键帧点云"
// 存入 next_layer.pcds[i]，作为下一层的输入
```

3.4 窗口的数学本质

全局 BA 的 Hessian 矩阵：



HBA 把大部分帧间耦合关在 10 帧的小窗口里，窗口间的弱耦合通过重叠和 PGO 传播间接处理。

四、体素 octree：平面特征提取

4.1 构建流程

```
int layer_limit = 2;  // 最多递归剖分 2 次
int MIN_PT = 15;      // 体素内最少点数阈值

// cut_voxel: 点云分配到体素
//   voxel_size 默认 4.0m
//   每个体素是一个 OCTO_TREE_ROOT

// recut: 对每个体素递归剖分
```

```
//   Layer 0:  voxel_size = 4m    → 4m³ 的体素
//   → cut_func(): 剖分为 8 个子体素 → 2m³
//   → cut_func(): 再剖分            → 1m³
//   → 到达 layer_limit=2，停止剖分
//   → judge_eigen(): 判断是否包含有效平面
```

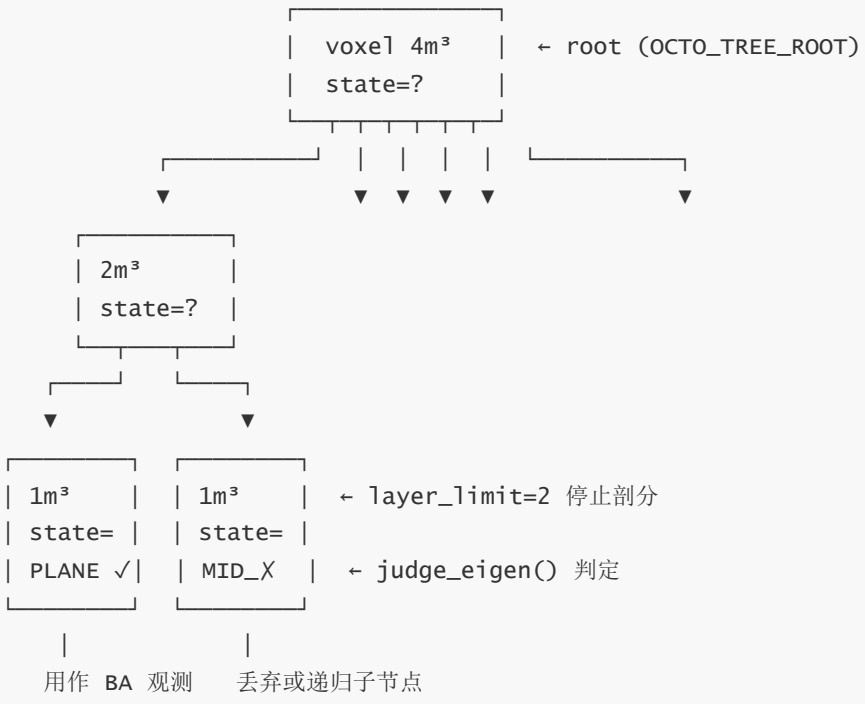
4.2 平面判定：judge_eigen()

```
// 对体素内所有点计算协方差矩阵，特征值 λ1 ≤ λ2 ≤ λ3
eigen_ratio = λ1 / λ3;   // 最小特征值 / 最大特征值

// 判定为 PLANE 的条件：
//   eigen_ratio < eigen_thr  (默认 0.1)
//   且子节点特征值满足一致性检验 (ratio between 1/9 and 4)

if (judge_eigen() == PLANE):
    该体素的点集构成有效平面 → 作为 BA 观测（残差 = λ1）
else:
    递归检查子节点，或标记为 MID_NODE 丢弃
```

4.3 体素树结构



五、分层金字塔：层间压缩与传播

5.1 分层的本质

分层的真正含义是**关键帧金字塔降采样**：

```
Layer 0:  1000 帧原始点云  ──滑动窗口BA──▶  200 个合并关键帧
Layer 1:  200 个关键帧      ──滑动窗口BA──▶  40 个合并关键帧
Layer 2:  40 个关键帧       ──global_BA──▶  全局优化后的位姿
```

每层的输出是下一层的输入，**帧数逐层约减半**（≈ WIN_SIZE/GAP = 10/5 = 2 倍压缩率）。

这**不是**树结构，因为：

- 层间是**单向压缩**（多帧 → 一帧），不是父子分支
- 每层只有**一个**输出给下一层
- 层内并行（16 线程），但层间是**串行 barrier**

类比：**逐层降采样的图像金字塔。**

5.2 层间衔接代码

```
// main() — hba.cpp 核心循环
for (int i = 0; i < total_layer_num - 1; i++) {
    // 处理 Layer i，输出到 Layer i+1
    distribute_thread(hba.layers[i], hba.layers[i+1]);
    // 将 Layer i 优化后的位姿传播到 Layer i+1
    hba.update_next_layer_state(i);
}

// 最后一层：全局 BA（无窗口，全部帧一起优化）
global_ba(hba.layers[total_layer_num - 1]);

// 最终：PGO 将优化结果传播回所有原始帧
hba.pose_graph_optimization();
```

5.3 update_next_layer_state 的作用

```
Layer 0 优化后:
  帧 0-9   → 合并为 Layer 1 的帧 0，位姿 = Win 0 优化后帧 0 的位姿
  帧 5-14  → 合并为 Layer 1 的帧 1，位姿 = Win 1 优化后帧 5 的位姿
  帧 10-19 → 合并为 Layer 1 的帧 2，位姿 = Win 2 优化后帧 10 的位姿
  ...

取每个窗口的第一帧位姿作为合并关键帧的初始位姿
```

5.4 global_ba（最顶层）

最顶层不再分窗口，全部帧联合优化：

- 使用 `eigen_ratio × 2`（更宽松，收敛更快）
- 所有 40 帧同时优化（240 维变量）
- 输出最终精化位姿

六、完整数据流（以 3 层为例）

```
┌──────────────────────────────────────────────────────────────────┐
│ Layer 0（最细层，1000 帧原始 PCD）                                  │
│                                                                  │
│  distribute_thread(layer0, layer1)                               │
│  ┌──────────────────────────────────────────────────┐            │
│  │ 16 个线程并行处理窗口                                │            │
│  │                                                  │            │
│  │ 线程0: Win[0..9]   → octree(voxel=4m, recutx2)       │            │
│  │        → 提取平面特征 → LM 优化 10 帧位姿 (60 变量)     │            │
│  │        → 合并 10 帧 → layer1.pcds[0]                 │            │
│  │                                                  │            │
│  │ 线程1: Win[5..14]  → ... → layer1.pcds[1]            │            │
│  │ 线程2: Win[10..19] → ... → layer1.pcds[2]            │            │
│  │ ...                                              │            │
│  └──────────────────────────────────────────────────┘            │
│                                                                  │
│  update_next_layer_state(0):                                     │
│    取 Win 第一帧的优化位姿作为 Layer1 各关键帧的初始位姿                │
└──────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│ Layer 1（中间层，200 个合并关键帧）                                  │
│                                                                  │
│  distribute_thread(layer1, layer2)                               │
│  ┌──────────────────────────────────────────────────┐            │
│  │ 同 Layer 0 的逻辑，但输入是 layer1.pcds[]              │            │
│  │ 窗口大小仍是 10，GAP 仍是 5                           │            │
│  │ → 输出 layer2.pcds[]（约 40 个更粗的关键帧）           │            │
│  └──────────────────────────────────────────────────┘            │
│                                                                  │
│  update_next_layer_state(1)                                      │
└──────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│ Layer 2（最粗层，40 个关键帧）                                      │
│                                                                  │
│  global_ba(layer2)  ← 不再分窗口!                                  │
│  ┌──────────────────────────────────────────────────┐            │
│  │ 所有 40 帧一起优化                                    │            │
```

```
|   | 一次性的全局 BA（eigen_ratio × 2，收敛更快）                  |            |
|   | 优化变量：40 × 6 = 240 维                                    |            |
|   └──────────────────────────────────────────────────────────┘            |
|                                                                          |
|  pose_graph_optimization()                                               |
|   ┌──────────────────────────────────────────────────────────┐            |
|   | 将 Layer 2 的优化结果作为约束                                |            |
|   | 通过 PGO 传播回 Layer 1 → Layer 0 的所有 1000 帧            |            |
|   └──────────────────────────────────────────────────────────┘            |
└──────────────────────────────────────────────────────────────────────────┘
```

七、复杂度分析：为什么比全局 BA 快

7.1 数值对比（1000 帧）

方法	Hessian 维度	求解代价（稀疏）	单次墙钟（估算）
全局 BA	6000 × 6000	$O(N^2) \approx 36M$	分钟 ~ 小时级
HBA 单窗口	60 × 60	$O(10^2) = 100$	毫秒级
HBA Layer 0 (200 窗口)	60 × 60 × 200 ÷ 16	~45k	秒级
HBA Layer 1 (40 窗口)	60 × 60 × 40 ÷ 16	~9k	亚秒级
HBA Layer 2 global_BA (40帧)	240 × 240	~58k	秒级
HBA 总计	—	**~112k**	**十秒级**

加速比：36M / 112k ≈ **~320×**（稀疏），实际因体素提取等开销会低一些，但仍约 2 个数量级。

7.2 为什么分窗就能快

BA 的 Hessian 矩阵中，全局 BA 的非对角块（不同帧之间的共视约束）使矩阵远非对角占优，Cholesky 分解时 fill-in 严重。HBA 把大部分耦合关在 10 帧的小窗口里，窗口之间的弱耦合通过窗口重叠（50%）和 PGO 传播间接处理。

关键洞察：**把一个大矩阵求逆（$O(N^2)$）拆成几百个小矩阵求逆（$O(W^2)$），并行求解，再用金字塔压缩帧数，最后 PGO 缝合。**窗口大小 W=10 是常数，所以整体复杂度从 $O(N^2)$ 降为 $O(N)$。

7.3 HBA 的额外加速手段

手段	效果
16 线程并行	200 个窗口同时求解，墙钟时间 ÷ 16
金字塔压缩	1000 帧 → 200 帧 → 40 帧，层层减半
octree 平面提取	只对判为 PLANE 的体素构建 Hessian，MID_NODE 丢弃
残差截断 (reject_ratio=5%)	每轮迭代扔掉 5% 最大残差体素，减少离群干扰
LM 阻尼迭代	自适应调整阻尼因子，通常 5-10 次迭代收敛

7.4 精度代价

	全局 BA	HBA
精度	理论最优	接近最优（论文声称"similar accuracy"）
窗口边界效应	无	有（50% 重叠 + PGO 缓解）
单次/需多次	一次即全局最优	只跑一次；不满意可重跑

八、关键参数表

参数	定义位置	默认值	含义
`total_layer_num`	launch	3	金字塔层数
`thread_num`	launch	16	并行线程数
`WIN_SIZE`	ba.hpp	10	局部 BA 窗口大小（帧数）
`GAP`	ba.hpp	5	相邻窗口步长（帧数）
`layer_limit`	ba.hpp	2	octree 最大递归剖分次数
`MIN_PT`	ba.hpp	15	体素内最少点数阈值
`voxel_size`	hba.hpp	4.0 m	初始体素尺寸
`eigen_ratio`	hba.hpp	0.1	平面判定阈值（$\lambda min/\lambda$max）
`reject_ratio`	hba.hpp	0.05	残差截断比例（鲁棒核）
`downsample_size`	hba.hpp	0.1 m	点云降采样尺寸

九、相关工作的完整谱系

```
BALM (RAL 2021, HKU MARS)
├── 首次提出 LiDAR Bundle Adjustment
|   用体素提取平面/边缘特征作为 BA 的观测
|   缺点：全局 BA 计算量 O(N³)，大场景不可用
|
├── BALM2 (2022)
|   增量式 BA，支持在线运行
|   缺点：长距离漂移累积后仍需全局优化
|
├── HBA (RAL 2023, HKU MARS)  ← 本文分析对象
|   分层金字塔 BA + PGO 传播
|   精度接近全局 BA，速度大幅提升
|   开源：github.com/hku-mars/HBA
|
```

```
├── LEMON-Mapping (2024, 浙大 FAST Lab)
|   多机器人协作场景的空间分层 BA
|   稠密回环区域 → Spatial HBA
|   稀疏回环区域 → Diffusion BA (DBA)
|   创新：loop recall 机制恢复漏检回环
|
├── DH-LC (IROS 2022, 三星)
|   视觉 SLAM 的 Hybrid BA
|   Incremental BA + Full BA 自适应切换
|
├── FAST-LIO-SLAM / Point-LIO (2024)
|   LIO 前端 + 回环 + PGO
|   回环检测：Scan Context / Intensity Scan Context
|   后端：GTSAM 因子图 PGO
```

	PGO	**HBA**
优化对象	仅关键帧位姿	位姿 + 点云几何一致性
优化变量数	几百个位姿	几百个位姿 + 若干体素平面
单次迭代代价	O(N³) N=位姿数	O(L·W³) L=层数 W=窗口大小
地图一致性	无法保证	直接优化
速度	秒级	十秒～分钟级
适用场景	实时在线	离线精化/后处理

十、源码关键函数速查

HBA 仓库结构

```
hku-mars/HBA/
├── include/
|   ├── ba.hpp       ← WIN_SIZE, GAP, VOX_HESS, OCTO_TREE, VOX_OPTIMIZER
|   ├── hba.hpp      ← voxel_size, eigen_ratio, reject_ratio, LAYER
|   ├── mypcl.hpp    ← 点云 I/O 工具
|   └── tools.hpp    ← 数学工具
├── source/
|   ├── hba.cpp      ← main(), cut_voxel(), parallel_comp(), global_ba()
|   ├── calculate_MME.cpp
|   └── visualize.cpp
└── launch/
    └── hba.launch   ← total_layer_num, thread_num
```

核心函数

函数	文件	职责
`main()`	hba.cpp	入口：按层循环 → distribute_thread → global_ba → PGO
`cut_voxel()`	hba.cpp	点云按 voxel_size 分配到体素，构建 OCTO_TREE_ROOT
`recut()`	ba.hpp	递归剖分 octree，judge_eigen 判定平面
`parallel_comp()`	hba.cpp	线程工作函数：滑动窗口 BA 优化
`parallel_tail()`	hba.cpp	尾部线程：同 parallel_comp + 尾部窗口 + 计时
`distribute_thread()`	hba.cpp	创建 16 线程，前 15 个跑 parallel_comp，最后 1 个跑 parallel_tail
`global_ba()`	hba.cpp	最顶层全局 BA（无窗口分割）
`update_next_layer_state()`	hba.hpp	层间位姿传播
`pose_graph_optimization()`	hba.hpp	PGO 全局融合
`damping_iter()`	ba.hpp	LM 优化求解器
`judge_eigen()`	ba.hpp	协方差特征值比判定平面
`remove_outlier()`	ba.hpp	残差排序截断（reject_ratio）

参考文献

- Liu X, Liu Z, Kong F, Zhang F. "Large-Scale LiDAR Consistent Mapping Using Hierarchical LiDAR Bundle Adjustment." *IEEE RA-L*, Vol. 8, No. 3, pp. 1523–1530, 2023.
- HBA 源码
- BALM / BALM2
- FAST-LIO2 — Xu et al., TRO 2022